

Sprawozdanie z etapu 5.

Szymon Hładyszewski, Wiktor Koczkodaj, Jakub Kogut, Wojciech Typer

1 lutego 2026

1 Dokumentacja

Dokumentacja endpointów REST API znajduje się pod adresem `wealthwise.pl/api/docs`. Szczegółowa dokumentacja została opisana w Sprawozdaniu nr 3. Została ona ściśle podtrzymna podczas dalszej implementacji.

2 Backend

Nasz backend będzie odpowiedzialny za przechowywanie oraz przetwarzanie wszelakich danych jak i musi posiadać *user friendly* metodę na ich wprowadzenie. W tym celu zdecydowaliśmy się na użycie frameworku **Django** wraz z **Django REST Framework** do stworzenia API. Do przechowywania danych użyjemy bazy danych **PostgreSQL**, która jest utrzymywana przez **Django**.

2.1 Baza danych

W bazie danych przechowywane będą wszelkie dane dotyczące użytkowników jak i zdobytych przez nich nagród, czy dane do kursów oraz quizów, a także forum.

2.1.1 Tabele

W bazie danych znajdować się będą następujące tabele:

- **User**: przechowuje dane użytkowników
 - **id (PK)** - unikalny identyfikator użytkownika
 - **nickname** - nazwa użytkownika *unique*
 - **password_hash** - zahashowane hasło użytkownika
 - **avatar_id** - identyfikator avatara użytkownika (możliwe rozszerzenie bazy o tabelę Avatar)
- **Course**: zawiera kursy dostępne w serwisie
 - **id (PK)** - unikalny identyfikator kursu
 - **name** - tytuł kursu
- **Lesson**: reprezentuje lekcje, z których składają się kursy
 - **id (PK)** - unikalny identyfikator lekcji
 - **name** - tytuł lekcji
- **Slide**: reprezentuje slajdy w lekcjach
 - **id (PK)** - unikalny identyfikator slajdu
 - **lesson_id (FK)** - identyfikator lekcji, do której należy slajd
 - **text_content** - treść slajdu
 - **order** - kolejność slajdu w lekcji
 - **image_url** - (opcjonalnie) URL do obrazka na slajdzie

- **Quiz:**
 - **id (PK)** - unikalny identyfikator quizu
 - **name** - tytuł quizu
 - **exp_weight** - waga doświadczenia za poprawne rozwiązanie quizu
- **Question:** pytania w quizach
 - **id (PK)** - unikalny identyfikator pytania
 - **quiz_id (FK)** - identyfikator quizu, do którego należy pytanie
 - **question_type** - typ pytania (otwarte/zamknięte)
 - **content** - treść pytania
- **AnswerPattern:** wzorce odpowiedzi dla otwartych pytań (wartości numeryczne)
 - **id (PK)** - unikalny identyfikator wzorca odpowiedzi
 - **question_id (FK)** - identyfikator pytania, do którego należy wzorec
 - **pattern** - wzorec odpowiedzi (np. zakres wartości)
- **AnswerOption:** możliwe odpowiedzi dla pytań zamkniętych
 - **id (PK)** - unikalny identyfikator opcji odpowiedzi
 - **question_id (FK)** - identyfikator pytania, do którego należy opcja
 - **content** - treść opcji odpowiedzi
 - **is_correct** - czy opcja jest poprawną odpowiedzią
- **UserProgress:** śledzi postępy użytkowników w kursach (zabezpieczenie przed ponownym zdobywaniem expa)
 - **id (PK)** - unikalny identyfikator postępu
 - **user_id (FK)** - identyfikator użytkownika
 - **course_id (FK)** - identyfikator kursu
 - **completion_date** - data i czas ukończenia kursu
- **Post:** posty na forum
 - **id (PK)** - unikalny identyfikator postu
 - **user_id (FK)** - identyfikator autora postu
 - **title** - tytuł postu
 - **content** - treść postu
 - **creation_date** - data i czas utworzenia postu
- **Tag:** tagi przypisane do postów
 - **id (PK)** - unikalny identyfikator tagu
 - **name** - nazwa tagu
- **Comment:** komentarze do postów na forum
 - **id (PK)** - unikalny identyfikator komentarza
 - **post_id (FK)** - identyfikator postu, do którego należy komentarz
 - **parent_comment_id (FK, nullable)** - identyfikator komentarza nadrzędnego (jeśli to odpowiedź)
 - **user_id (FK)** - identyfikator autora komentarza
 - **content** - treść komentarza
 - **creation_date** - data i czas utworzenia komentarza
- **Reaction:** tabela reakcji (upvoty/reakcje emoji) do postów i komentarzy
 - **id (PK)** - unikalny identyfikator reakcji
 - **reaction_type** - typ reakcji (np. upvot, emoji)
- **UserStreak:** do przechowywania danych o długości streaków użytkowników¹
 - **user_id (PK, FK)** - identyfikator użytkownika

¹można by rozważyć czy nie przechowywać to w tabeli User, ale wydaje się to bardziej czytelne

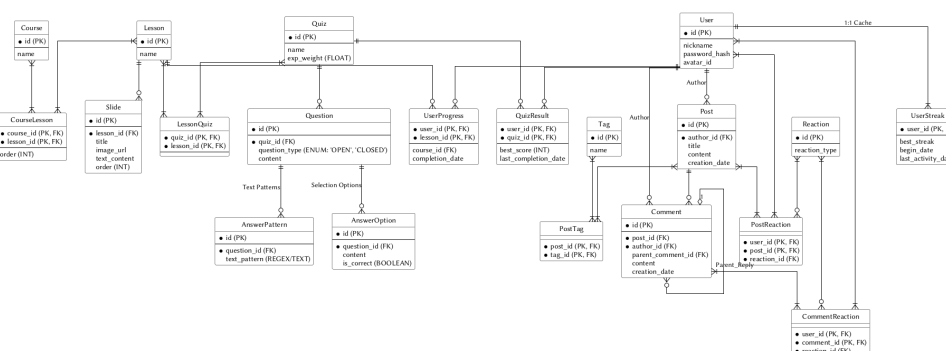
- **best_streak** - aktualny streak użytkownika (najdłuższy)
- **begin_date** - data rozpoczęcia aktualnego streaku
- **last_activity_date** - data ostatniej aktywności użytkownika

Oraz wymagane tabele łączące:

- **QuizResult:** przechowuje wyniki quizów użytkowników
 - user_id (PK, FK) - identyfikator użytkownika
 - quiz_id (PK, FK) - identyfikator quizu
 - best_score - najlepszy wynik uzyskany w quizie
 - last_completion_date - data i czas ukończenia quizu
- **CourseLesson:** łącząca kursy z lekcjami
 - course_id (PK, FK) - identyfikator kursu
 - lesson_id (PK, FK) - identyfikator lekcji
 - order - kolejność lekcji w kursie
- **LessonQuiz:** łącząca lekcje z quizami
 - lesson_id (PK, FK) - identyfikator lekcji
 - quiz_id (PK, FK) - identyfikator quizu
- **PostTag:** łącząca posty z tagami
 - post_id (PK, FK) - identyfikator postu
 - tag_id (PK, FK) - identyfikator tagu
- **PostReaction:** łącząca posty z reakcjami
 - post_id (PK, FK) - identyfikator postu
 - reaction_id (PK, FK) - identyfikator reakcji
 - user_id (PK, FK) - identyfikator użytkownika, który dodał reakcję
- **CommentReaction:** łącząca komentarze z reakcjami
 - comment_id (PK, FK) - identyfikator komentarza
 - reaction_id (PK, FK) - identyfikator reakcji
 - user_id (PK, FK) - identyfikator użytkownika, który dodał reakcję

Staraliśmy się, aby baza danych była znormalizowana do trzeciej postaci normalnej (3NF), co pozwoli uniknąć redundancji danych oraz zapewnić integralność referencyjną.

Poniżej znajduje się diagram ERD przedstawiający strukturę bazy danych oraz relacje między tabelami.



Rysunek 1: Diagram ERD bazy danych

2.1.2 Funkcje, Triggery

Zaplanowaliśmy również kilka funkcji oraz triggerów w bazie danych, które będą automatyzowały pewne procesy:

- Funkcja do aktualizacji streaku użytkownika w tabeli **UserStreak**. Przy każdym logowaniu należy sprawdzić datę ostatniej aktywności i jeżeli było to dzień wcześniej, zwiększyć aktualny streak o 1. Jeżeli nie było, to ustawić datę `begin_date` oraz `last_activity_date` na dzisiejszą datę i zresetować.
Przy wykonaniu quizu należy ustawić `last_activity_date` na dzisiejszą datę i sprawdzić czy streak jest dłuższy niż `best_streak`, jeżeli tak to zaktualizować.
- Trigger po dodaniu rekordu do tabeli **UserProgress**, który będzie nadawał użytkownikowi odpowiednią ilość expa na podstawie wagi kursu.
- Odpowiednio utrzymywać dane w tabelach związanych z **Comment/Post** przy usuwaniu postów/komentarzy (kaskadowe usuwanie lub ustawianie na NULL w przypadku komentarzy nadrzędnych)

2.2 Permisje

Aby zabezpieczyć aplikację przed nieautoryzowanym dostępem, wprowadzimy system permisji. Użytkownicy będą podzieleni na dwie grupy:

- **Użytkownicy zarejestrowani**² - będą mieli dostęp do kursów, quizów oraz forum, nagród innych użytkowników
- **Administratorzy** - będą mieli dostęp do interfejsu administratora, gdzie będą mogli zarządzać kursami, quizami oraz użytkownikami

Dodatkowo, jeżeli użytkownik chciałby edytować/usunąć swój post/komentarz na forum, będzie wymagane sprawdzenie czy jest on autorem danego postu/komentarza.

3 Interfejs administratora

Django posiada wbudowany interfejs administratora, który pozwala na zarządzanie bazą danych bez konieczności tworzenia własnego panelu. W naszym projekcie wykorzystamy ten interfejs do zarządzania w szczególności kursami oraz quizami.

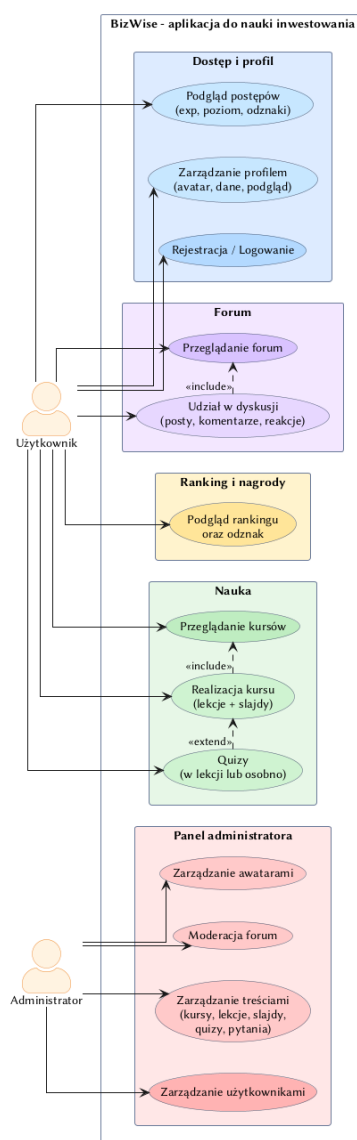
Administrator będzie miał możliwość tworzenia oraz edytowania poprzez interfejs administratora:

- tworzenie lekcji na podstawie dodawania slajdów z tekstem, ewentualnie obrazkami
- tworzenie quizów na podstawie pytań otwartych (wartości numeryczne) oraz zamkniętych.
- dodawanie avatarów użytkowników (użytkownicy mogą wybierać avatary z puli dostępnych w serwisie)

4 Diagramy

Poniżej przedstawiamy diagramy przedstawiające strukturę projektu:

²restrakcja poprzez nickname i hasło



Rysunek 2: Diagram przypadków użycia

5 Aplikacja Mobilna

Front-end naszego systemu zostanie zrealizowany jako natywna aplikacja mobilna na system Android. Aplikacja będzie stanowiła główny interfejs dla użytkownika, umożliwiając interakcję z danymi dostarczonymi przez API backendowe. Zgodnie z założeniami, do jej stworzenia wykorzystamy język **Kotlin** oraz nowoczesny toolkit UI – **Jetpack Compose**.

5.1 Główne moduły i ekrany

Struktura nawigacji aplikacji została podzielona na kilka kluczowych obszarów funkcjonalnych:

5.1.1 Moduł Autentykacji

Odpowiada za zarządzanie dostępem do aplikacji.

- **Ekran Startowy:** Pierwszy widok witający użytkownika, pozwalający na wybór między logowaniem a rejestracją.
- **Ekran Rejestracji/Logowania:** Formularze umożliwiające wprowadzenie danych (nickname, hasło). Moduł ten będzie obsługiwał walidację danych wejściowych po stronie klienta oraz bezpieczne przechowywanie otrzymanego od backendu tokena autoryzacyjnego.

5.1.2 Moduł Profilu Użytkownika

Centrum informacji o postępach gracza.

- Wyświetlanie awatara oraz nazwy użytkownika.
- Prezentacja statystyk: aktualny **streak** oraz historia zdobytych punktów doświadczenia.
- Historia ukończonych kursów i quizów.

5.1.3 System Nauki (Learning Path)

Rdzeń aplikacji, realizujący przepływ wiedzy od kursu do sprawdzenia wiedzy.

- **Lista Kursów:** Ekran prezentujący dostępne kursy wraz z paskiem postępu.
- **Ekran Lekcji i Slajdów:** Widok szczegółowy lekcji. Slajdy będą wyświetlane sekwencyjnie. Aplikacja będzie obsługiwać renderowanie tekstu oraz ładowanie obrazów z sieci.
- **Przepływ (Flow):** Użytkownik będzie mógł przejść do następnego slajdu/lekcji dopiero po zapoznaniu się z obecnym, co wymusza liniowy proces nauki.

5.1.4 Moduł Quizów

Interfejs do weryfikacji wiedzy, obsługujący logikę pytań zdefiniowanych w bazie danych.

- **Ekran Quizu:** Dynamiczny widok obsługujący dwa typy pytań:
 - *Zamknięte:* Lista opcji wyboru na podstawie danych z bazy.
 - *Otwarte:* Pola tekstowe/numeryczne.
- **Ekran Wyników:** Po zakończeniu quizu aplikacja wyśle odpowiedzi do API, a następnie wyświetli uzyskany wynik, zdobytą liczbę punktów doświadczenia oraz informację o ewentualnej aktualizacji streaku.
- **Tryb Samodzielnej Nauki:** Osobny ekran pozwalający na uruchomienie losowych quizów bez powiązania z konkretną lekcją, w celu utrwalenia wiedzy.

5.1.5 Forum Dyskusyjne

Społecznościowy aspekt aplikacji.

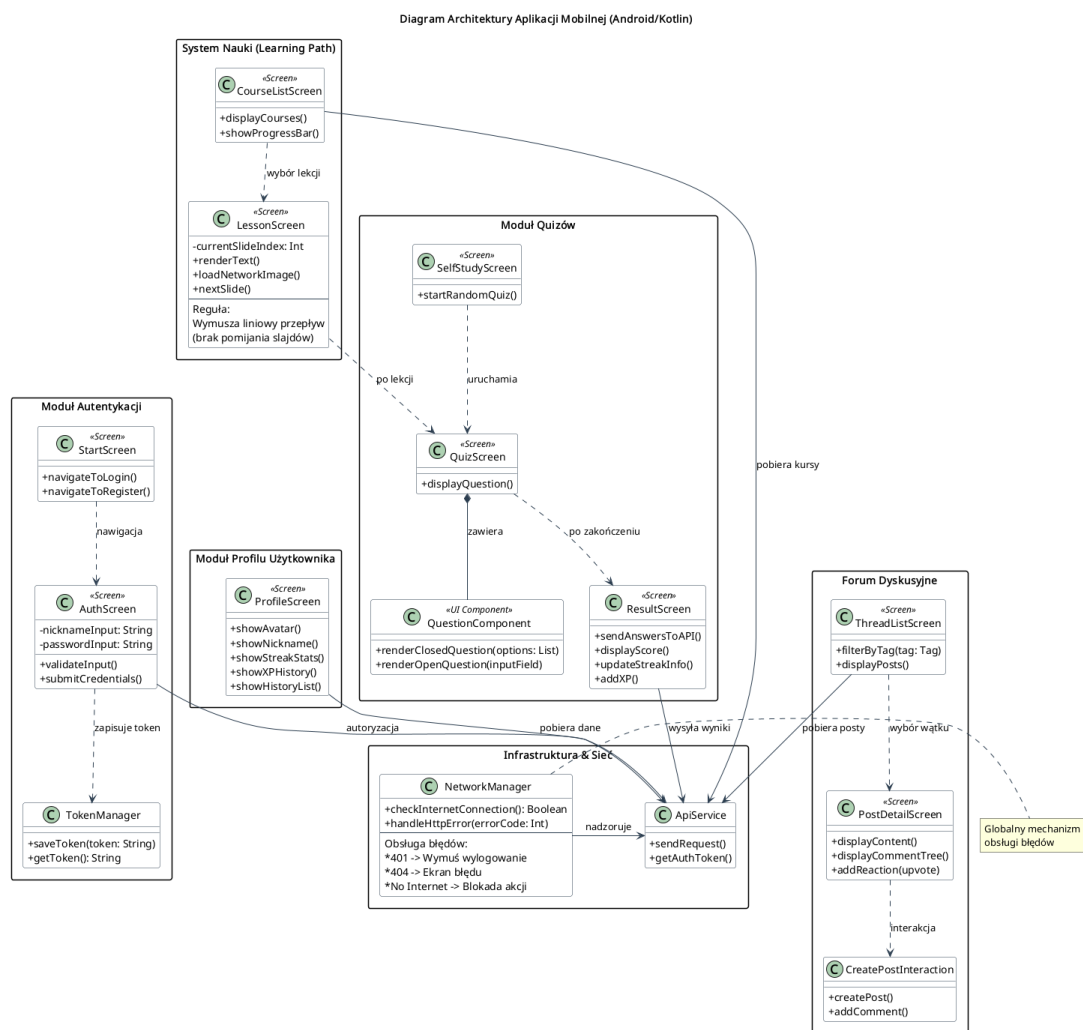
- **Lista Wątków:** Wyświetlanie postów (**Post**) z możliwością filtrowania po tagach (**Tag**).

- **Szczegóły Posta:** Widok pojedynczego wątku wraz z drzewiastą strukturą komentarzy (**Comment**).
- **Interakcje:** Możliwość dodawania nowych postów, komentowania oraz dodawania reakcji (upvote) do treści innych użytkowników.

5.2 Komunikacja i obsługa błędów

Aplikacja zostanie wyposażona w globalny mechanizm obsługi błędów:

- **Brak połączenia z internetem:** Wyświetlenie odpowiedniego komunikatu i zablokowanie akcji wymagających synchronizacji z API.
- **Obsługa błędów HTTP:** Interpretacja kodów błędów (np. 401 Unauthorized - wymuszenie wylogowania, 404 Not Found - odpowiedni ekran błędu).



Rysunek 3: Diagram UML dla aplikacji mobilnej

6 Raport z testów

Testy uruchomiono poleceniem `python manage.py test`. Wykonywane były testy automatyczne aplikacji (testy jednostkowe i integracyjne) zdefiniowane w projekcie. Konfiguracja testowa korzysta z bazy SQLite (`config/settings_test.py`), co umożliwia szybkie uruchamianie scenariuszy bez zależności zewnętrznych. Oczekiwany wynik było pomyślne zakończenie całego zestawu testów bez błędów i niepowodzeń.

6.1 Struktura backendu

Backend to projekt Django z wyodrębnionymi aplikacjami domenowymi:

- **users** – konta, profile, awatary, odznaki oraz logika streaków,
- **courses** – kursy, lekcje, slajdy, postępy użytkowników,
- **quizzes** – quizy, pytania, odpowiedzi i wyniki,
- **forum** – posty, komentarze i reakcje,
- **pages** – widoki publiczne i szablony,
- **config** – ustawienia projektu (w tym ustawienia testowe).

6.2 Zakres testów

Moduł users (15 testów)

1. **Logowanie z poprawnymi danymi:** wysłanie loginu i hasła zwraca kod 200 oraz tokeny `access` i `refresh`.
2. **Logowanie bez nazwy użytkownika:** brak pola `username` zwraca kod 400 i błąd walidacji.
3. **Rejestracja:** poprawny payload tworzy użytkownika i zwraca tokeny.
4. **Profil me:** odpowiedź zawiera podstawowe pola profilu (m.in. `username`, `avatar`, `exp`, `streak`).
5. **Lista awatarów:** endpoint zwraca listę dostępnych awatarów.
6. **Ukończone kursy:** endpoint `me/progress` zawiera identyfikatory ukończonych kursów.
7. **Aktualizacja awatara:** zmiana awatara zapisuje się w profilu użytkownika.
8. **Najlepszy streak bez rekordu:** brak danych zwraca `best_streak = 0`.
9. **Bieżący streak:** długość streaku obliczana jest na podstawie dat aktywności.
10. **Odznaki użytkownika:** endpoint `me/badges` zwraca przypisane odznaki.
11. **EXP z quizów:** `exp` jest sumą procentów najlepszych wyników z wagą `exp_weight`.
12. **Odznaka za 100% w quizie:** przyznawana jest `BADGE_QUIZ_MASTER`.
13. **Odznaka za pierwszy ukończony kurs:** przyznawana jest `BADGE_FIRST_COURSE`.
14. **Odznaka za komentarz:** przyznawana jest `BADGE_HELPING_HAND`.
15. **Odznaki za streak:** dla streaku 14 dni przyznawane są `BADGE_STREAK_7` i `BADGE_LOYAL_USER`.

Moduł courses (3 testy)

1. **Blokada lekcji:** po wejściu w kurs odblokowana jest tylko pierwsza lekcja, kolejne są zablokowane.
2. **Ukończenie lekcji i kursu:** ukończenie dwóch lekcji tworzy postęp lekcji i końcowy postęp kursu.
3. **Szczegóły lekcji:** endpoint zwraca slajdy lekcji oraz identyfikator powiązanego quizu.

Moduł quizzes (6 testów)

1. **Lista quizów:** endpoint zwraca wszystkie utworzone quizy.
2. **Szczegóły quizu bez flagi poprawności:** odpowiedzi nie zawierają pola `is_correct`.
3. **Wysłanie odpowiedzi:** poprawne odpowiedzi skutkują wynikiem 100%, zapisem `best_score` i naliczeniem `exp_gained`.
4. **Leaderboard i wyniki:** leaderboard zawiera użytkownika z najlepszym wynikiem, a endpoint wyników zwraca `best_score`.
5. **Autoryzacja klucza odpowiedzi:** brak uwierzytelnienia zwraca 401.
6. **Klucz odpowiedzi (zamknięte i otwarte):** zwracane są poprawne opcje dla pytań zamkniętych oraz wzorzec liczbowy dla pytań otwartych.

Moduł forum (2 testy)

1. **Podsumowanie reakcji:** endpoint zwraca liczbę reakcji typu LIKE dla posta.
2. **Edycja komentarza:** autor może zaktualizować treść komentarza (HTTP 200).

6.3 Wyniki testów

Wszystkie testy zakończyły się **pomyślnie**. Testy potwierdzają poprawne działanie:

- endpointów API i podstawowych scenariuszy użytkownika,
- mechanizmów uwierzytelniania i autoryzacji,
- logiki przyznawania odznak oraz wyliczania postępów,
- relacji między modelami (kursy–lekcje–postępy, quizy–wyniki, forum–reakcje).

Brak błędów w testach potwierdza stabilność systemu oraz gotowość do dalszego rozwoju.